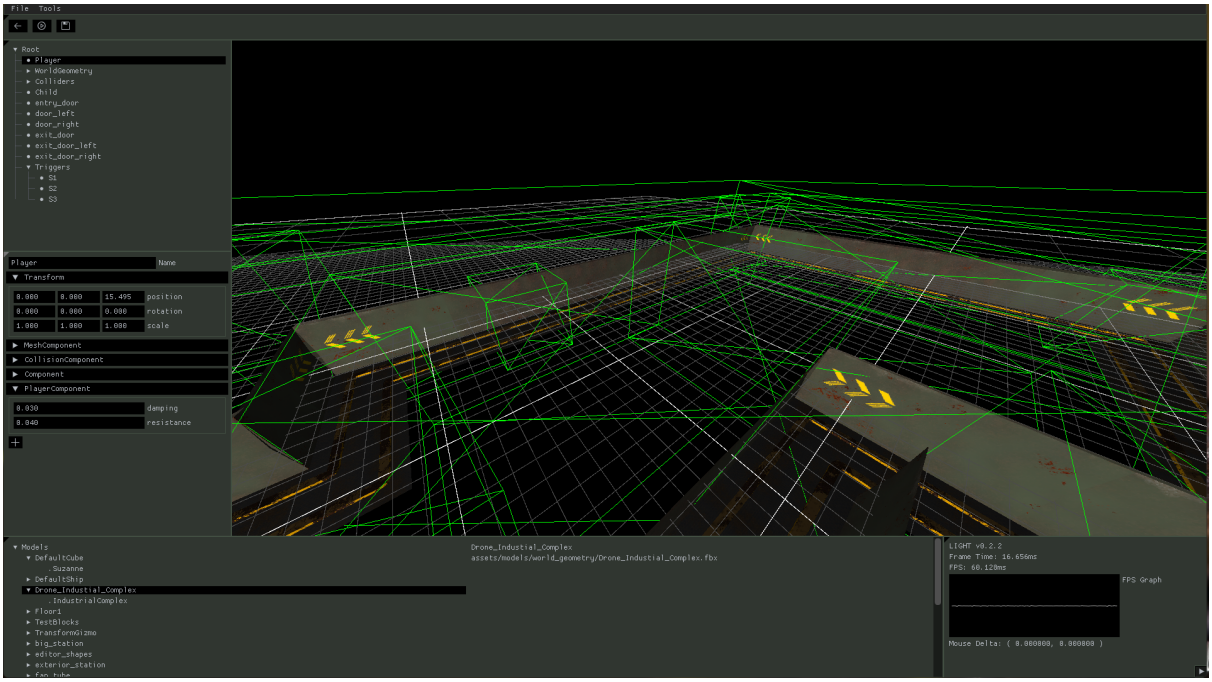
n#+TITLE: Light Game Engine Quality Management Plan

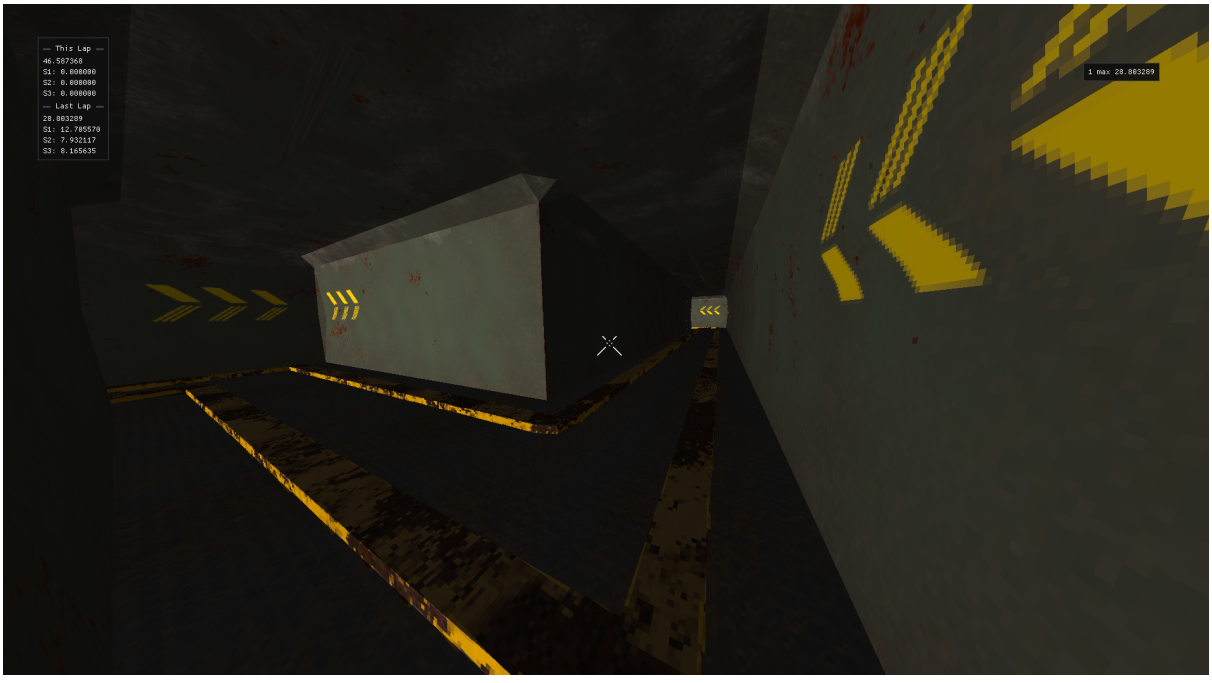
1 Project

1.1 Light

Light is a minimalist game engine, designed to allow a modern, editor-based workflow while being lightweight enough to run on the vast majority of modern hardware. It was written over a period of about 20 days for CSC455 at SUNY Oswego. It is available, fully open-source and free to use, at `https://git.deepfrog.org/deepfrog/light`.



(a) The engine in editor mode, being used to construct a scene.



(a) The same scene in gameplay mode.

1.2 Component Under Test

The focus of the testing will be limited to the ‘Transform‘ class due to time constraints. This class is used to build a graph structure representing a 3D scene, along with allowing pieces of C++ code (‘Component‘s) to be attached to nodes in this graph. The vast majority of the other features of the engine rely on this class, so it is the most critical to test. Additionally, it is known to have at least one flaw.

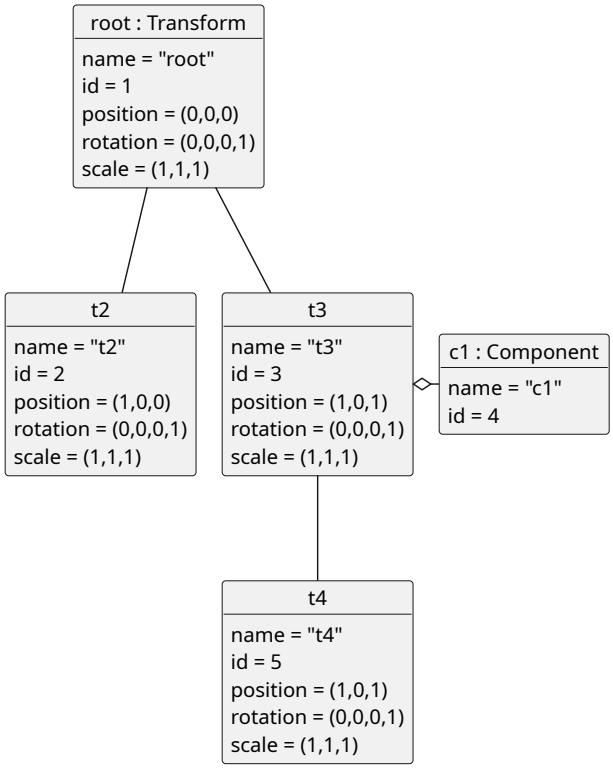
2 The Transform Class

2.1 Purpose

The Transform class forms a single node in a rooted, directed graph that represents a three-dimensional scene. Each transform represents the location, orientation, and scale of a coordinate frame (in mathematical terms, it defines a basis with an offset). The methods being tested here are the ones that allow constructing this graph, as well as the ones that interact with the Transform's spatial properties (position, rotation, and scale).

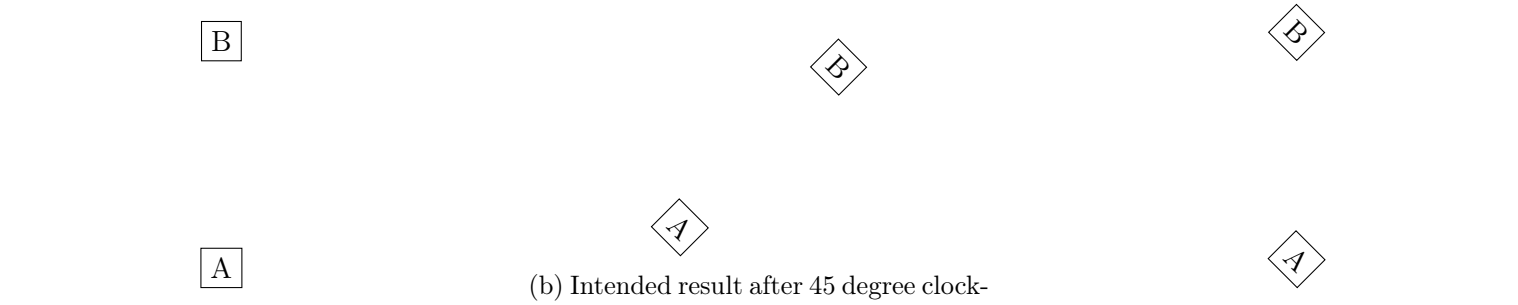
2.2 Object Diagram

This object diagram provides an example of what a transform graph might look like during a run of the program. Fields irrelevant to testing have been omitted.



2.3 Currently Known Flaw

To illustrate the flaw, imagine a transform called "A" at the origin, with a transform "B" as its child, translated along the Y axis an arbitrary distance. If the transform "A" is rotated about its Z axis, the child should follow the parent, such that it maintains its position and rotation relative to the parent. Instead, the child rotates independently.



(a) Initial setup, A is the parent of B

(b) Intended result after 45 degree clockwise rotation of A

(c) Actual result with current code

3 Quality Management Plan

3.1 Quality Criteria

Quality Property	Definition	Metric	Criterion
Functional Correctness	Test cases yield the expected results anticipated during derivation.	Number of test cases that pass.	90% of test cases pass.
Fault Tolerance	Software should run as intended even if invalid inputs are supplied.	Number of test cases that produce an unrecoverable error, such as a segfault.	All test cases properly recover from invalid inputs.
Performance	Software should run efficiently enough to allow smooth rendering.	Time taken to update state and render a frame.	Average frame time is under 16ms (60 frames per second or higher)

3.2 Quality Management Tools

Tool	Purpose
Doxygen	Automatic generation of well-formatted documentation from source code comments.
Mermaid	Generation of control flow graphs from text definitions.
PlantUML	Generation of UML diagrams from text definition.
Doctest	Automated unit testing and result reporting.

3.3 Test Plan

Testing Phase	Testing Activity	Test Deadline
Specification-Based Testing	Derivation of equivalence classes, boundary values, and test cases based on the specification of each method.	October 11th, 2026

Further phases are yet to be determined.

4 Equivalence Class Testing

4.1 Transform::AddChild(Transform* tr)

Specification

```
/**
 * Add a child transform to this transform
 *
 * If the desired child is already a child of this transform
 * log a warning and ignore it
 *
 * @param tr the transform to add as a child
 *
 */
void AddChild(Transform *tr);

```

Equivalence Classes

Eq. Class ID	Param	Description	Representative
4.1.EC1.1	tr	a valid pointer	new Transform()
4.1.EC1.a	tr	a null pointer	nullptr

Test Cases

Test Case ID	Param. *tr*	Expected	Coverage
4.1.TC1	A pointer to some valid Transform	No output	4.1.EC1.1
4.1.TC2	nullptr	Error: log message	4.1.EC1.a

Test Implementation

```
#include "../doctest/doctest.h"
#include "../engine/transform.h"
#include "../engine/component.h"
using namespace engine;

TEST_CASE("4.1.TC1") {
  Transform* test_transform = new Transform();
  Transform* child_transform = new Transform();
  //redirect cout to a string so we can check for the error message
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  test_transform->AddChild(child_transform);
  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() == ""); //no output expected
}
TEST_CASE("4.1.TC2") {
  Transform* test_transform = new Transform();
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  test_transform->AddChild(nullptr);
  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() != ""); //some kind of log output expected
}
```

4.2 Transform::AddComponent(Component* component)

Specification

```

/**
 * Add a component to this transform
 *
 * If the supplied pointer is null, log error and ignore
 * If the supplied pointer is not a component, log error and ignore
 *
 * @param component A pointer to the component to add
 */
void AddComponent(Component* component);
```

Equivalence Classes

Eq. Class ID	Param	Description	Representative
4.2.EC1.1	component	a valid pointer to a class that extends Component	new Component()
4.2.EC1.a	component	a valid pointer to something other than a Component	new Transform()
4.2.EC1.b	component	an invalid pointer	nullptr

Test Cases

Test Case ID	Param. *tr*	Expected	Coverage
4.2.TC1	new TestComponent()	No output	4.2.EC1.1
4.2.TC2	new Transform()	Disallowed by type checking, TC implementation omitted	4.2.EC1.a
4.2.TC3	nullptr	Error: log message and ignore	4.2.EC1.b

Test Implementation

```
TEST_CASE("4.2.TC1") {
  Transform* test_transform = new Transform();
  Component* test_component = new Component();
  //redirect cout to a string so we can check for the error message
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  test_transform->AddComponent(test_component);
  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() == ""); //no output expected
}
TEST_CASE("4.2.TC3") {
  Transform* test_transform = new Transform();
  Component* test_component = nullptr;
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  test_transform->AddComponent(test_component);
  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() != ""); //some kind of log output expected
}
```

4.3 Transform::FindChildByName(std::string name)

Equivalence Classes

Eq. Class ID	Param	Description	Representative
4.3.EC1.1	name	A string	"string"
4.3.EC1.2	name	The empty string	""
4.3.EC1.a	name	Not a string	1

Test Cases

Test Case ID	Param. *name*	Expected	Coverage
4.3.TC1	"string"	nullptr	4.3.EC1.1
4.3.TC2	""	nullptr	4.3.EC1.2
4.3.TC3	1	This will be disallowed by type checking, so TC implementation is omitted.	4.3.EC1.a

- The specification is unclear on whether the empty string is a valid name for a Transform, so I am assuming it is a valid transform name.

Test Implementation

```
TEST_CASE("4.3.TC1") {
  Transform* test_transform = new Transform();
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  Transform* out = test_transform->FindChildByName("string");
  std::cout.rdbuf(old_cout);
  CHECK(out == nullptr);
  delete test_transform;
}
TEST_CASE("4.3.TC2") {
  Transform* test_transform = new Transform();
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  Transform* out = test_transform->FindChildByName("");
  std::cout.rdbuf(old_cout);
  CHECK(out == nullptr);
  delete test_transform;
}
```

4.4 Transform::FindComponentByUID(uint64$_t$ id)

Equivalence Classes

Eq. Class ID	Param	Description	Representative
4.4.EC1.1	id	A positive integer	1
4.4.EC1.2	id	Zero	0
4.4.EC1.a	id	A negative integer	-1

Boundary Value Analysis

Parameter	Boundary Values
id	0, UINT64_MAX

Test Cases

Test Case ID	Param. *id*	Expected	Coverage
4.4.TC1	1	nullptr	4.4.EC1.1
4.4.TC2	0	nullptr	4.4.EC1.2
4.4.TC3	-1	Disallowed by type checking, TC implementation omitted	4.4.EC1.a

Boundary Value Test Cases

Test Case ID	Param. *id*	Expected	Note
4.4.TC4	1	nullptr	Already covered by 4.4.TC1
4.4.TC5	0	nullptr	Already covered by 4.4.TC2
4.4.TC6	-1	Disallowed by type checking, TC implementation omitted	Already covered by 4.4.TC3
4.4.TC7	UINT64_MAX-1	nullptr	
4.4.TC8	UINT64_MAX	nullptr	
4.4.TC9	UINT64_MAX+1	same as 4.4.TC5 due to overflow	

Test Implementation

```
TEST_CASE("4.4.TC1") {
  Transform* test_transform = new Transform();
  Component* out = test_transform->FindComponentByUID(1);
  CHECK(out == nullptr);
  delete test_transform;
}
TEST_CASE("4.4.TC7") {
  Transform* test_transform = new Transform();
  Transform* out = test_transform->FindTransformByUID(UINT64_MAX-1);
  CHECK(out == nullptr);
  delete test_transform;
}
TEST_CASE("4.4.TC2") {
  Transform* test_transform = new Transform();
  Component* out = test_transform->FindComponentByUID(0);
  CHECK(out == nullptr);
  delete test_transform;
}
TEST_CASE("4.4.TC7") {
  Transform* test_transform = new Transform();
  Component* out = test_transform->FindComponentByUID(UINT64_MAX-1);
  CHECK(out == nullptr);
  delete test_transform;
}
TEST_CASE("4.4.TC8") {
  Transform* test_transform = new Transform();
  Component* out = test_transform->FindComponentByUID(UINT64_MAX);
  CHECK(out == nullptr);
  delete test_transform;
}
TEST_CASE("4.4.TC9") {
  Transform* test_transform = new Transform();
  Component* out = test_transform->FindComponentByUID(UINT64_MAX+1);
  CHECK(out == nullptr);
  delete test_transform;
}
```

4.5 Transform::FindTransformByUID(uint64$_t$ id)

Equivalence Classes

Equivalence classes same as Transform::FindComponentByUID.

Test Cases

Test Case ID	Param. *tr*	Expected	Coverage
4.5.TC1.1	1	nullptr	4.5.EC1.1
4.5.TC1.2	0	nullptr	4.5.EC1.2
4.5.TC1.a	-1	Disallowed by type checking, TC implementation omitted	4.5.EC1.a

Boundary Value Analysis

Boundary values and additional test cases are the same as 4.4.

Test Implementation

```
TEST_CASE("4.5.TC1") {
  Transform* test_transform = new Transform();
  Transform* out = test_transform->FindTransformByUID(1);
  CHECK(out == nullptr);
  delete test_transform;
}
TEST_CASE("4.5.TC2") {
  Transform* test_transform = new Transform();
  Transform* out = test_transform->FindTransformByUID(0);
  CHECK(out == nullptr);
  delete test_transform;
}
TEST_CASE("4.5.TC7") {
  Transform* test_transform = new Transform();
  Transform* out = test_transform->FindTransformByUID(UINT64_MAX-1);
  CHECK(out == nullptr);
  delete test_transform;
}
TEST_CASE("4.5.TC8") {
  Transform* test_transform = new Transform();
  Transform* out = test_transform->FindTransformByUID(UINT64_MAX);
  CHECK(out == nullptr);
  delete test_transform;
}
TEST_CASE("4.5.TC9") {
  Transform* test_transform = new Transform();
  Transform* out = test_transform->FindTransformByUID(UINT64_MAX+1);
  CHECK(out == nullptr);
  delete test_transform;
}

```

4.6 Transform::GetComponent(std::string name)

Equivalence Classes

Possible issues with specification:

- The method to get a transform by name is called ‘FindChildByName‘, this method should be updated to match that scheme as ‘FindComponentByName‘.

  The equivalence classes are the same as Transform::FindChildByName.

Test Cases

Test Case ID	Param. *name*	Expected	Coverage
4.6.TC1	"string"	nullptr	4.6.EC1.1
4.6.TC2	""	nullptr	4.6.EC1.2
4.6.TC3	1	This will be disallowed by type checking, so TC implementation is omitted.	4.6.EC1.a

Test Implementation

```
TEST_CASE("4.6.TC1") {
  Transform* test_transform = new Transform();
  Component* out = test_transform->GetComponent("string");
  CHECK(out == nullptr);
  delete test_transform;
}
TEST_CASE("4.6.TC2") {
  Transform* test_transform = new Transform();
  Component* out = test_transform->GetComponent("");
  CHECK(out == nullptr);
  delete test_transform;
}
```

4.7 Transform::GetGlobalPosition()

Equivalence Classes

Takes no parameters, so equivalence class testing is not appropriate.

4.8 Transform::GetGlobalRotation()

Equivalence Classes

Takes no parameters, so equivalence class testing is not appropriate.

4.9 Transform::GetGlobalScale()

Equivalence Classes

Takes no parameters, so equivalence class testing is not appropriate.

4.10 Transform::Rotate(glm::vec3 axis, float amount)

Equivalence Classes

Eq. Class ID	Param	Description	Representative
4.10.EC1.1	axis	axis is a vector with nonzero length	{1.0,0.0,0.0}
4.10.EC1.a	axis	axis is a vector with zero length	{0.0,0.0,0.0}
4.10.EC1.b	axis	axis is not a vector	1
4.10.EC2.1	amount	amount is a float	90.0f
4.10.EC2.2	amount	amount is an integer	90
4.10.EC2.a	amount	amount is not a number	"90"

Test Cases

Test Case ID	Param.		Expected	Coverage
	axis	*amount*		
4.10.TC1	{1.0,0.0,0.0}	90.0f	No output	4.10.EC1.1 4.10.EC2.1
4.10.TC2	{1.0,0.0,0.0}	90	No output	4.10.EC2.2
4.10.TC3	{0.0,0.0,0.0}	90.0f	Error: log message	4.10.EC1.a
4.10.TC4	1	90.0f	Disallowed by type checker	4.10.EC1.b
4.10.TC5	{1.0,0.0,0.0}	"90"	Disallowed by type checker	4.10.EC2.a

Test Implementation

```
TEST_CASE("4.10.TC1") {
  Transform* test_transform = new Transform();
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  test_transform->Rotate(glm::vec3(1.0,0.0,0.0), 90.0f);
  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() == ""); //no output expected
  delete test_transform;
}
TEST_CASE("4.10.TC2") {
  Transform* test_transform = new Transform();
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  test_transform->Rotate(glm::vec3(1.0,0.0,0.0), 90);
  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() == ""); //no output expected
  delete test_transform;
}
TEST_CASE("4.10.TC3") {
  Transform* test_transform = new Transform();
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  test_transform->Rotate(glm::vec3(0.0,0.0,0.0), 90.0f);
  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() != ""); //Error log output expected
  delete test_transform;
}
```

4.11 Transform::Scale(glm::vec3 amount)

Equivalence Classes

Eq. Class ID	Param	Description	Representative
4.11.EC1.1	amount	Any vector other than the zero vector or the identity vector of component-wise multiplication	{x=2.0,y=3.0,z=2.0}
4.11.EC1.2	amount	The identity element of component-wise multiplication	{x=1.0,y=1.0,z=1.0}
4.11.EC1.3	amount	The zero vector	{x=0.0,y=0.0,z=0.0}
4.11.EC1.a	amount	Not a vector	1

Test Cases

Test Case ID	Param. *amount*	Expected	Coverage
4.11.TC1	{2.0,3.0,2.0}	No output	4.11.EC1.1
4.11.TC2	{1.0,1.0,1.0}	No output	4.11.EC1.2
4.11.TC3	{0.0,0.0,0.0}	No output	4.11.EC1.3
4.11.TC4	1	Disallowed by type checker	4.11.EC1.a

Test Implementation

```
TEST_CASE("4.11.TC1") {
  Transform* test_transform = new Transform();
  glm::vec3 test_vector = glm::vec3(2.0,3.0,2.0);
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  test_transform->Scale(test_vector);
  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() == ""); //no output expected
  delete test_transform;
}
TEST_CASE("4.11.TC2") {
  Transform* test_transform = new Transform();
  glm::vec3 test_vector = glm::vec3(1.0,1.0,1.0);
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  test_transform->Scale(test_vector);
  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() == ""); //no output expected
  delete test_transform;
}
TEST_CASE("4.11.TC3") {
  Transform* test_transform = new Transform();
  glm::vec3 test_vector = glm::vec3(0.0,0.0,0.0);
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  test_transform->Scale(test_vector);
  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() == ""); //no output expected
  delete test_transform;
}
```

4.12 Transform::Translate(glm::vec3 offset)

Equivalence Classes

Eq. Class ID	Param	Description	Representative
4.12.EC1.1	amount	Any vector that is not the identity vector	{x=2.0,y=3.0,z=2.0}
4.12.EC1.2	amount	The identity vector for addition	{x=0.0,y=0.0,z=0.0}
4.12.EC1.a	amount	Not a vector	1

Test Cases

Test Case ID	Param. *amount*	Expected	Coverage
4.12.TC1	{2.0,3.0,2.0}	No output	4.12.EC1.1
4.12.TC2	{0.0,0.0,0.0}	No output	4.12.EC1.2
4.12.TC3	1	Disallowed by type checker	4.12.EC1.a

Test Implementation

```
TEST_CASE("4.12.TC1") {
  Transform* test_transform = new Transform();
  glm::vec3 test_vector = glm::vec3(2.0,3.0,2.0);
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  test_transform->Translate(test_vector);
  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() == ""); //some kind of log output expected
  delete test_transform;
}
TEST_CASE("4.12.TC2") {
  Transform* test_transform = new Transform();
  glm::vec3 test_vector = glm::vec3(0.0,0.0,0.0);
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());
  test_transform->Translate(test_vector);
  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() == ""); //some kind of log output expected
  delete test_transform;
}
```

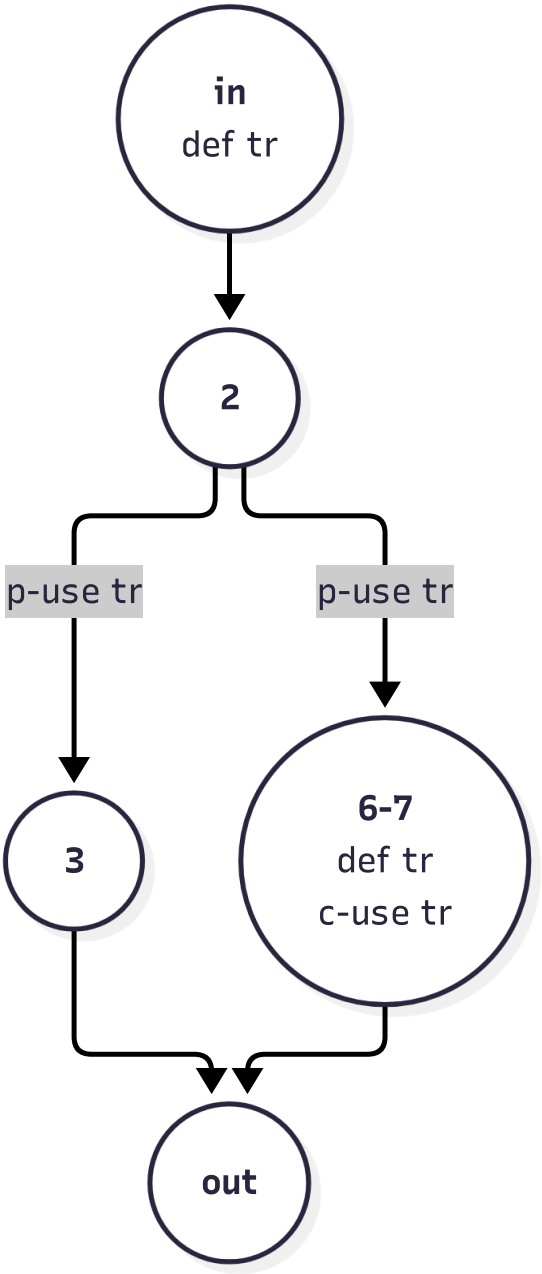
5 Source Code Based Testing

5.1 Transform::AddChild(Transform* tr)

Source Code

```
void Transform::AddChild(Transform *tr) {
  if (tr == nullptr) {
    std::cout << "Error: null transform passed to Transform::AddChild\n";
    return;
  }
  tr->parent = this;
  this->children.push_back(tr);
}
```

Control Flow Graph



Test Cases

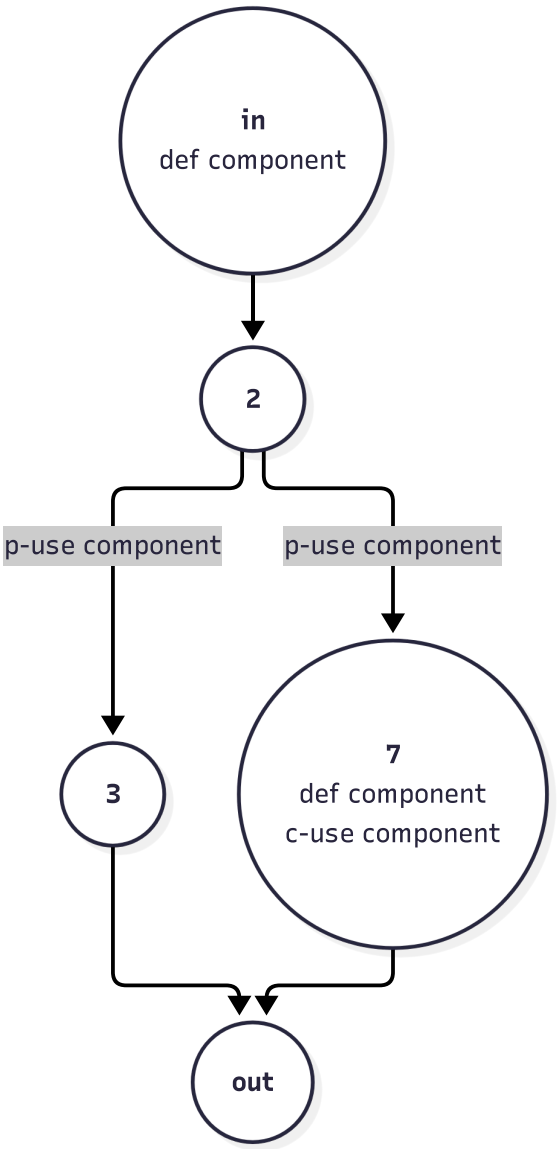
ID	tr	Path
5.1.TC1	nullptr	(in, 2, 3, out)
5.2.TC2	new Transform()	(in, 2, 6-7,out)

5.2 Transform::AddComponent(Component* component)

Source Code

```
void Transform::AddComponent(Component *component) {
  if (component == nullptr)
  {
    std::cout << "Error: null component passed to Transform::AddComponent\n";
    return;
  }
  component->SetOwner(this);
  this->components.push_back(component);
}
```

Control Flow Graph



Test Cases

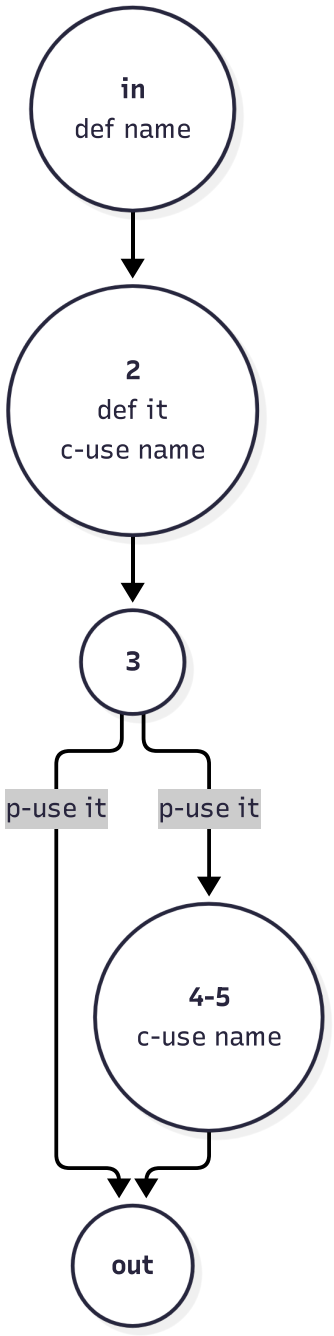
ID	component	Path
5.2.TC1	nullptr	(in,2,4,out)
5.2.TC2	new Component()	(in,2,7,out)

5.3 Transform::FindChildByName(std::string name)

Source Code

```
Transform *Transform::FindChildByName(std::string name) {
  auto it = std::find_if(children.begin(), children.end(), [name](Transform* t) {return t->name == name;})
  if (it == children.end()) {
    std::cout << "Supplied name " + name + " not found on " +
                 this->name << "\n";
    return nullptr;
  } else {
    return *it;
  }
}
```

Control Flow Graph



Test Cases

ID	name	Path
5.3.TC1		

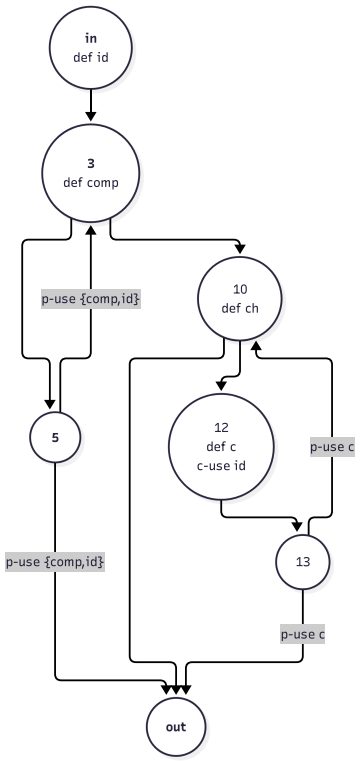
5.4 Transform::FindComponentByUID(uint64$_t$ id)

Source Code

```
Component* Transform::FindComponentByUID(uint64_t id)
{
  for(auto comp : components)
  {
    if(comp->GetUID() == id)
    {
      return comp;
```

```
    }
  }
  for(auto ch : children)
  {
    Component* c = ch->FindComponentByUID(id);
    if(c!=nullptr)
    {
      return c;
    }
  }
  return nullptr;
}
```

Control Flow Graph



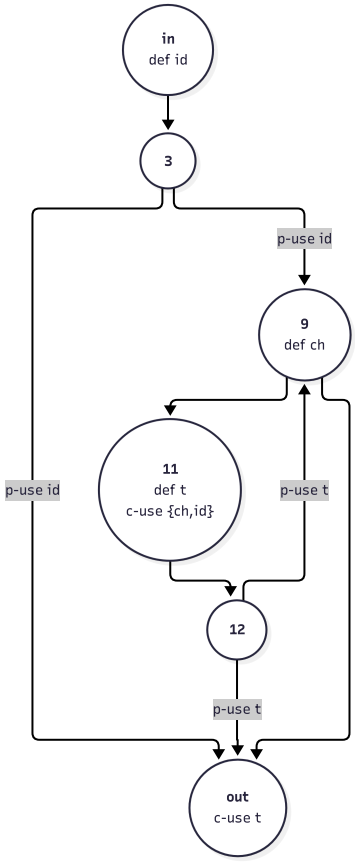
5.5 Transform::FindTransformByUID(uint64$_t$ id)

Source Code

```
Transform* Transform::FindTransformByUID(uint64_t id)
{
  if(this->GetUID() == id)
  {
    return this;
  }
  else
  {
    for(auto ch : children)
    {
      Transform* t = ch->FindTransformByUID(id);
      if(t != nullptr)
      {
        return t;
      }
```

```
    }
    return nullptr;
  }
}
```

Control Flow Graph

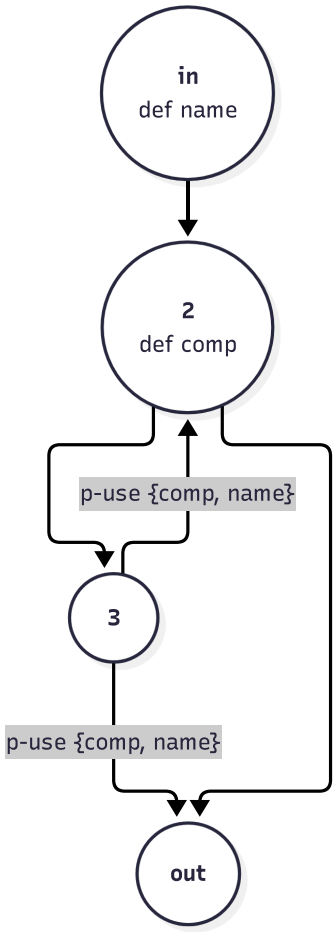


5.6 Transform::GetComponent(std::string name)

Source Code

```
Component *Transform::GetComponent(std::string name) {
  for (auto comp : components) {
    if (comp != nullptr && comp->name == name) {
        return comp;
    }
  }
  return nullptr;
}
```

Control Flow Graph

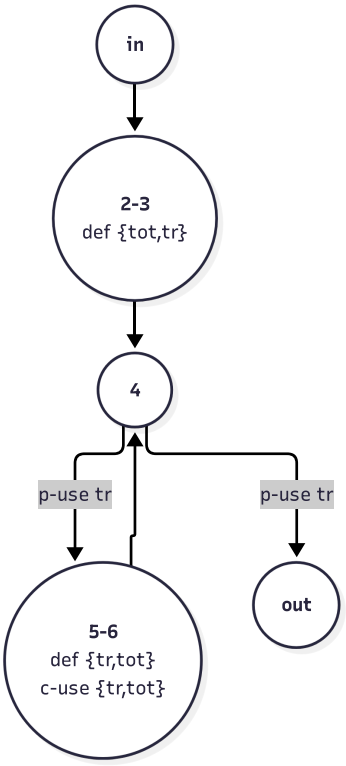


5.7 Transform::GetGlobalPosition()

Source Code

```
glm::vec3 Transform::GetGlobalPosition() {
  glm::vec3 tot(0.0f, 0.0f, 0.0f);
  Transform *tr = this;
  while (tr != nullptr) {
    tot += tr->GetLocalPosition();
    tr = tr->parent;
  }
  return tot;
}
```

Control Flow Graph

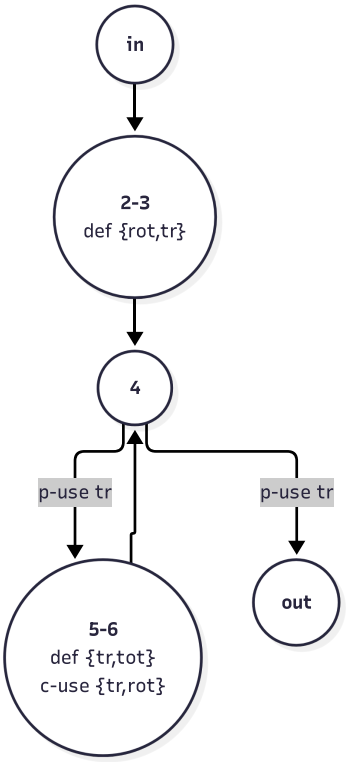


5.8 Transform::GetGlobalRotation()

Source Code

```
glm::quat Transform::GetGlobalRotation() {
  glm::quat rot = glm::identity<glm::quat>();
  Transform *tr = this;
  while (tr != nullptr) {
    rot = tr->GetLocalRotation() * rot;
    tr = tr->parent;
  }
  return rot;
}
```

Control Flow Graph

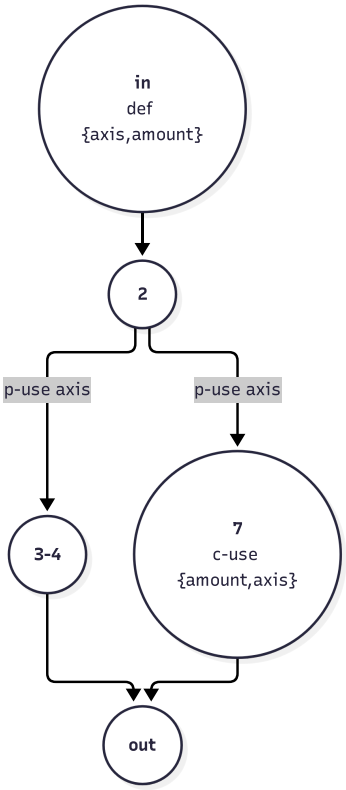


5.9 Transform::GetGlobalScale()

Source Code

```
glm::vec3 Transform::GetGlobalScale() {
  glm::vec3 tot(1.0f, 1.0f, 1.0f);
  Transform *tr = this;
  while (tr != nullptr) {
    tot *= tr->GetLocalScale();
    tr = tr->parent;
  }
  return tot;
}
```

Control Flow Graph

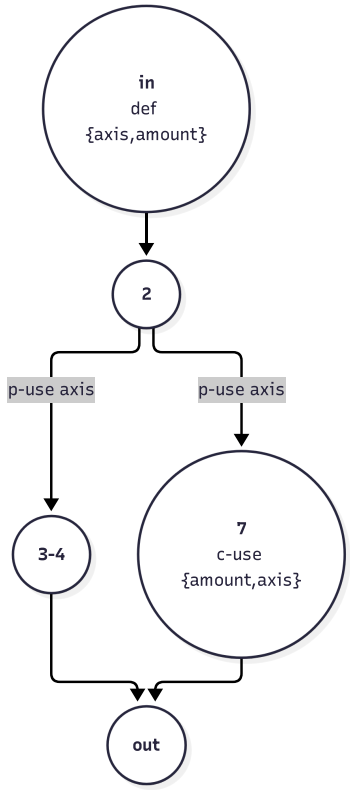


5.10 Transform::Rotate()

Source Code

```
void Transform::Rotate(glm::vec3 axis, float amount) {
  if (glm::length(axis) == 0) {
    std::cout << "Error: the zero vector does not constitute a valid axis for "
                 "Transform::Rotate\n";
    return;
  }
  this->rotation = glm::rotate(this->rotation,amount,axis);
}
```

Control Flow Graph



Test Cases

ID	axis	amount	Path
5.10.TC1	glm::vec3(0,0,0)	10.0f (free)	(in,2,3-4,out)
5.10.TC2	glm::vec3(1,0,0)	10.0f (free)	(in,2,7,out)

6 Test Results

Running the suite of test cases described in this document yields the following output.

```
[doctest] doctest version is "2.5.0"
[doctest] run with "--help" for options
===============================================================================
[doctest] test cases: 27 | 27 passed | 0 failed | 0 skipped
[doctest] assertions: 27 | 27 passed | 0 failed |
[doctest] Status: SUCCESS!
```

This indicates that the system is working as intended with regard to its handling of valid and invalid inputs.